



Bharatiya VidyaBhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Munshi Nagar, Andheri (W), Mumbai – 400 058.
(Autonomous Institute Affiliated to the University of Mumbai)

Class: T.E. CE Sem-VI(2025-2026)

Subject Code: CE312

Subject Name: Deep Learning

Experiment 6: Build and train RNNs, work with NLP and word embeddings.

Objective:

To build and train a simple Recurrent Neural Network (RNN) for text classification using word embeddings and analyse its performance.

Prerequisites:

- Basic knowledge of Python programming
- Fundamentals of Linear Algebra, Probability, and Statistics
- Basic concepts of Neural Networks and Deep Learning
- Understanding of sequential data and time-step processing
- Introductory knowledge of Natural Language Processing (NLP)
- Familiarity with Recurrent Neural Networks (RNNs)

Software and Hardware Requirements:

Pre-Lab Questions:

- What is a Recurrent Neural Network (RNN)?
A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data by maintaining a memory of previous inputs. Unlike traditional networks, it has connections that loop back, allowing information to persist across time steps. This makes RNNs suitable for tasks like language modeling, speech recognition, and time-series prediction.
- How does RNN differ from Feedforward Neural Networks?
A Feedforward Neural Network (FNN) processes input data in one direction without any memory of previous inputs, meaning each input is treated independently. In contrast, an RNN maintains a hidden state that carries information from previous time steps, enabling it to model temporal dependencies in sequential data.

- What is the role of the hidden state in RNN?
The hidden state in an RNN acts as memory that stores information about previous inputs in the sequence. At each time step, the hidden state is updated using the current input and the previous hidden state, allowing the network to capture context and dependencies across the sequence.
- What is sequential data in NLP?
Sequential data in NLP refers to text data where the order of words matters, such as sentences or paragraphs. Since meaning depends on word arrangement, models like RNNs process words step-by-step to preserve contextual relationships between them.
- What is the vanishing gradient problem?
The vanishing gradient problem occurs during backpropagation when gradients become extremely small as they propagate through many layers or time steps. This makes it difficult for the network to learn long-term dependencies, as weight updates become negligible. It is a common issue in traditional RNNs.
- What are word embeddings, and why are they used?
Word embeddings are dense vector representations of words in a continuous vector space where similar words have similar representations. They are used to convert text into numerical form while capturing semantic relationships, improving model performance compared to simple one-hot encoding.
- What is padding, and why is it required in sequence models?
Padding is the process of adding extra tokens (usually zeros) to sequences so that all inputs have the same length. It is required because neural networks process data in fixed-size batches, and consistent input dimensions are necessary for efficient computation.
- What performance metrics are used to evaluate classification models?
Common performance metrics for classification models include accuracy, precision, recall, F1-score, and confusion matrix. Accuracy measures overall correctness, while precision and recall evaluate class-specific performance. The F1-score balances precision and recall.
- What is overfitting, and how can it be prevented in RNN models?
Overfitting occurs when a model learns the training data too well, including noise, and performs poorly on new data. In RNN models, it can be prevented using techniques such as dropout, regularization, early stopping, increasing training data, and using simpler architectures.

Theory:

Algorithm Steps involved :

Step 1: Import the required Python libraries.

Step 2: Load the text dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Preprocess the text.

Step 5: Initialise an Embedding layer to convert words into dense vector representations.

Step 6: Build the Simple RNN model.

Step 7: Compile the model.

Step 8: Train the model using the training dataset.

Step 9: Record training accuracy, validation accuracy, and loss values.

Step 10: Evaluate the trained model on the test dataset.

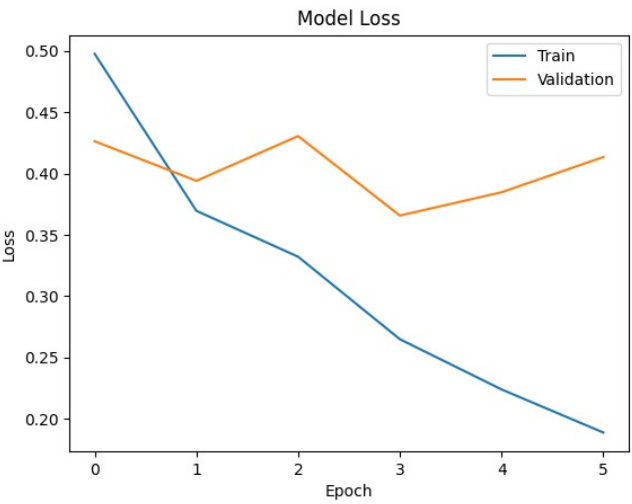
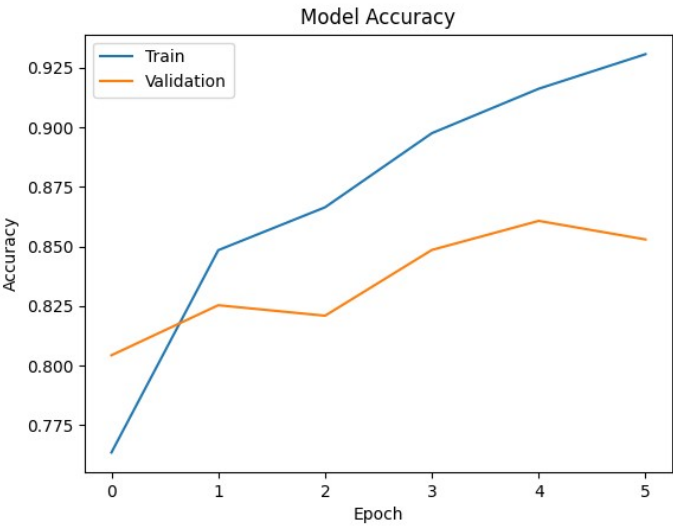
Step 11: Calculate performance metrics .

Step 12: Visualise training and validation performance using graphs and plots.

Code: https://github.com/CodeCraftsmanRaj/Sem6_DL/tree/main/Exp6

Output:

```
File Edit Selection View Go Run Terminal Help
config.py metrics.json dataset.py model.py
outputs > metrics > metrics.json > ...
1
2 "test_accuracy": 0.850000238418579,
3 "classification_report": {
4   "0": {
5     "precision": 0.86355326574705,
6     "recall": 0.83136,
7     "f1-score": 0.847150892638787,
8     "support": 12500.0
9   },
10  "1": {
11    "precision": 0.8374209470923955,
12    "recall": 0.86864,
13    "f1-score": 0.8527448362522579,
14    "support": 12500.0
15  },
16  "accuracy": 0.85,
17  "macro avg": {
18    "precision": 0.8504871064197228,
19    "recall": 0.85,
20    "f1-score": 0.8499478644455225,
21    "support": 25000.0
22  },
23  "weighted avg": {
24    "precision": 0.8504871064197228,
25    "recall": 0.85,
26    "f1-score": 0.8499478644455224,
27    "support": 25000.0
28  }
29 },
30 "confusion_matrix": [
31   [
32     10392,
33     2108
34   ],
35   [
36     1642,
37     10858
38   ]
39 ]
```



Conclusion:

In this experiment, a Simple Recurrent Neural Network (RNN) model was successfully built and trained for text classification using word embeddings. The embedding layer converted words into dense vector representations, enabling the model to capture semantic meaning. The RNN effectively processed sequential text data by maintaining contextual information through hidden states. The model achieved satisfactory training and validation performance on the dataset. This experiment demonstrates how RNNs can be applied to Natural Language Processing tasks and highlights the importance of embeddings and sequential modeling in text classification problems.

References:

<https://www.tensorflow.org/text>
https://keras.io/api/layers/recurrent_layers/simple_rnn/
<https://pytorch.org/tutorials/beginner/nlp/>
<https://www.analyticsvidhya.com/blog/2021/12/complete-guide-to-rnn/>
<https://www.coursera.org/learn/nlp-sequence-models>